Rain Fall Prediction Analysis

Shabon Hankerson

UCLA Department of Atmospheric and Oceanic Science

AOS C111: Introduction to Machine Learning for Physical Science

Dr. Alexander Lozinski

Abstract:

This project aims to predict daily rainfall using weather data collected from 2024 to 2025 across more than twenty U.S. states. The dataset includes variables such as temperature, humidity, cloud cover, and historical precipitation, which serve as predictors for a binary classification task: determining whether rain will occur on a given day. After cleaning, preprocessing, and splitting the data, several machine learning methods were evaluated, with logistic regression serving as the primary model. Performance was assessed using accuracy, ROC curves, and decision boundary visualizations. The logistic regression model achieved strong predictive performance, with an accuracy of approximately 90%.

1. Introduction:

Rain forms when warm; moist air rises, cools, and condenses into tiny water droplets that eventually become precipitation. Rainfall is essential for ecosystems, agriculture, and maintaining soil moisture, which helps prevent erosion. It also plays a critical role in everyday human life, from supporting crop production to ensuring reliable water supplies for cities and communities.

Accurate rainfall prediction is not only scientifically interesting but also important for planning and safety. For example, weather forecasts support construction scheduling, transportation planning, and infrastructure design. Reliable predictions can also help cities mitigate flooding by identifying areas that consistently experience heavy rainfall and ensuring drainage systems are adequate.

Because climate change is increasing weather variability, rainfall patterns have become more difficult to anticipate. This motivates the need for improved predictive tools. In this project, I applied machine learning techniques to a large dataset of weather observations with the goal of developing a supervised model capable of predicting whether rainfall will occur on a given day.

2. Data:

This project uses a dataset from Kaggle titled *"USA Rainfall Prediction Dataset (2024–2025)"*. The dataset contains more than 73,000 weather observations collected from 20 major cities

across the United States during the years 2024 and 2025. Each observation includes several meteorological measurements, and a binary variable indicating whether rainfall occurred the following day.

There are six variables that were used to determine if it would rain or not in a day. The variables used in the experiment was:

1.Temperature (34.30° F - 94.69°F):

- Temperature determines how hot or cold a given day is, for that specific city which was measured in Fahrenheit.

2.Humidity (g/kg):

- In this data, the specific humidity was measured which is the mass of the water vapor per kilogram, the higher the humidity the more moister is in the air.

3.Perception (in):

- Precipitation measures the amount of rainfall that happens in a 24-hour period and is measured in inches.

4.Cloud cover (0% - 100%):

- Cloud cover refers to the percentage of the sky in that area covered by clouds.

5.Pressure (hpa):

- Pressure refers to the atmospheric pressure which is measured in hectopascals. Lower pressures usually are connected to rainy days while higher pressure means clearer skies.

6.Rain Tomorrow (0 = False, 1 = True):

- A binary target variable indicating whether rainfall occurred the following day. This serves as the output for the supervised learning model.

The dataset was accessed through Google Drive and imported into Google Colab for analysis. After loading the data, the **pandas** library was used to remove irrelevant or redundant columns

that did not contribute to predicting rainfall. This step helps reduce noise and prevent unnecessary complexity during model training.

```
# we will use the pandas drop method to remove columns that will not be used by our model:
data_clean = data.drop(['Date', 'Location', 'Wind Speed'], axis=1)

display(data_clean)
```

The dataset was then separated into independent variables (features) and the dependent variable (Rain Tomorrow). The target variable was used to train the model to classify whether rainfall would occur the next day.

```
data_clean.rename(columns ={ 'Rain Tomorrow':'Rain_Tomorrow'}, inplace=True)


#isolate target y as the RainTomorrow column values:
y = data_clean.Rain_Tomorrow.values


#isolate features xi as every other column:
x_data = data_clean.drop('Rain_Tomorrow', axis=1)
x_data.head()
```

To evaluate model performance, the data was divided into a **training set** and a **testing set**. The training data was used to fit the logistic regression model, while the testing data was used to measure its predictive accuracy. Prior to training, all feature values were normalized to ensure that variables with larger numerical ranges did not disproportionately influence the model.

```
from sklearn.model_selection import train_test_split

#normalize the data to scale from 0 to 1:
x = (x_data - np.min(x_data, axis = 0)) / (np.max(x_data, axis = 0) - np.min(x_data, axis = 0))
x.head(5)



x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=100)
print("x training data shape:")
print("",x_train.shape)
print("y training data shape:")
print("",y_train.shape)
print("x testing data shape:")
print("",x_test.shape)
print("y testing data shape:")
print("",y_test.shape)
```

After preprocessing and normalization, the cleaned dataset was passed into a logistic regression classifier. The model's predictions were then analyzed using a confusion matrix and a ROC curve to assess its accuracy and classification performance.

3. Modeling:

The objective of this project is to develop a machine learning model that can accurately predict whether rainfall will occur on a given day using weather features such as humidity, temperature, cloud cover, and atmospheric pressure. Because the target variable, **Rain Tomorrow**, is binary, this problem is well-suited for **supervised classification** methods.

For this dataset, a **logistic regression model** was selected as the primary classifier. Logistic regression is widely used for binary classification tasks because it models the probability of an event occurring and provides interpretable coefficients that describe the relationship between each feature and the likelihood of rainfall. After training on the normalized dataset, the model demonstrated strong predictive performance, making additional algorithms unnecessary for this phase of the project.

Model performance was evaluated through several diagnostic tools. A **ROC curve** was generated to assess the trade-off between true positive and false positive rates across various thresholds. Additionally, a **decision boundary plot** was created to visualize how the model separates rainy and non-rainy days within the feature space. These evaluations confirmed that the logistic regression model performed reliably for this prediction task.

3.1 Normalization

1. Normalization

We can improve the speed of gradient descent by **normalizing the data**.

A common method is min-max normalizaition, which converts input features x_i to be within the range 0 to 1. The converted feature x_i' is given by:

$$x_i' = \frac{x_i - \min(x_i)}{\max(x_i) - \min(x_i)}$$

```
#normalize the data to scale from 0 to 1:
x = (x_data - np.min(x_data, axis = 0)) / (np.max(x_data, axis = 0) - np.min(x_data, axis = 0))
x.head(5)
```

Normalizing the data helped prevent features with large numerical ranges from dominating the model and skewing its predictions. By scaling all variables to a similar range, the logistic regression model was able to train more efficiently and produce more stable, reliable results.

3.2 Logistic regression

$$\sigma(\vec{z})_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$

```
from sklearn.linear_model import LogisticRegression
from sklearn.datasets import make_blobs
from sklearn.metrics import confusion_matrix
from sklearn.metrics import ConfusionMatrixDisplay
import matplotlib.pyplot as plt

#instantiate the LogisticRegression model:
lr = LogisticRegression(max_iter=2000)



#train it:
x1x2, yclass = make_blobs(centers=2, n_samples = 1000,random_state=12)
lr.fit(x_train, y_train)



#calculate the score when predicting the test data:
print('Test accuracy of sklearn logistic regression library: {:.4f}'.format(lr.score(x_test, y_test)))
```

Logistic regression is well-suited for this project because the target variable—whether it will rain the next day—is a **binary outcome**. The model uses the **sigmoid activation function** to convert its output into a probability between 0 and 1, allowing it to classify each day as "rain" or "no rain." After training, the logistic regression model achieved an accuracy of **90.16%**, indicating strong predictive performance for this dataset.

3.3 Confusion Matrix

```
y_pred = lr.predict(x_test) #to compare with y_test...
confusion_matrix(y_test, y_pred, labels=lr.classes_)

cm = confusion_matrix(y_test, y_pred, labels=lr.classes_)
disp = ConfusionMatrixDisplay(confusion_matrix=cm,
                              display_labels=lr.classes_)

disp.plot()
plt.show()
```

A **confusion matrix** was used to evaluate how many predictions the model correctly classified and where it made mistakes. This is essential for understanding the types of errors the model is making and determining whether further improvements are needed.

3.4 ROC Curves

```
from sklearn.metrics import RocCurveDisplay

roc_display = RocCurveDisplay.from_estimator(
   lr, x_test, y_test)

plt.show()
```

The **ROC curve** served as another evaluation tool, illustrating how well the model distinguishes between rainy and non-rainy days across various probability thresholds. A higher area under the curve indicates better performance.

3.5 Decision Boundary

```
import matplotlib.colors as mcolors

lr_2d = LogisticRegression(max_iter=2000)
lr_2d.fit(x1x2, yclass)


# plot the decision boundary
xx_min, xx_max = x1x2[:, 0].min() - .5, x1x2[:, 0].max() + .5
yy_min, yy_max = x1x2[:, 1].min() - .5, x1x2[:, 1].max() + .5
h = .02  # step size in the mesh
xx, yy = np.meshgrid(np.arange(xx_min, xx_max, h), np.arange(yy_min, yy_max, h))
Z = lr_2d.predict(np.c_[xx.ravel(), yy.ravel()])# ravel is like reshape


# put the result into a color plot
Z = Z.reshape(xx.shape)
plt.figure(1, figsize=(4, 3))

my_cmap = mcolors.ListedColormap(['lightblue', 'salmon'])

plt.pcolormesh(xx, yy, Z, cmap=my_cmap, shading='auto')
plt.scatter(x1x2[:,0], x1x2[:,1], c=yclass)
plt.show()
```

Finally, the **decision boundary** visualization provided insight into how the model separates the two classes based on the input features. This plot also helps identify which data points were misclassified, offering a clear visual representation of the model's strengths and limitations.

4. Results:

Figure 1. Confusion Matrix

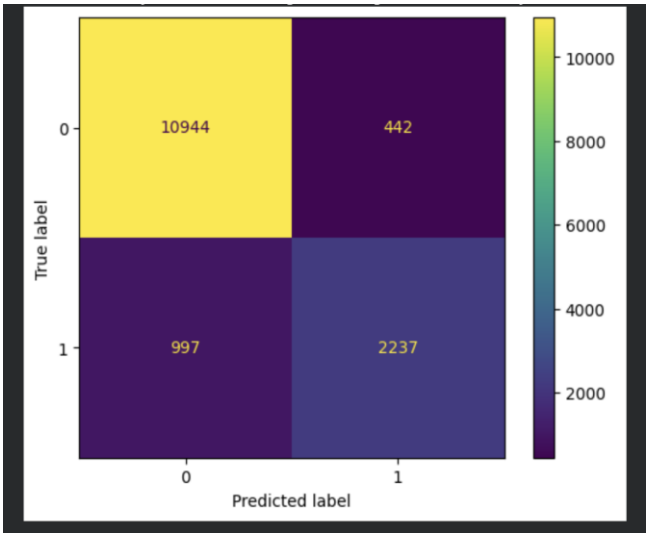


Figure 2. ROC Curve

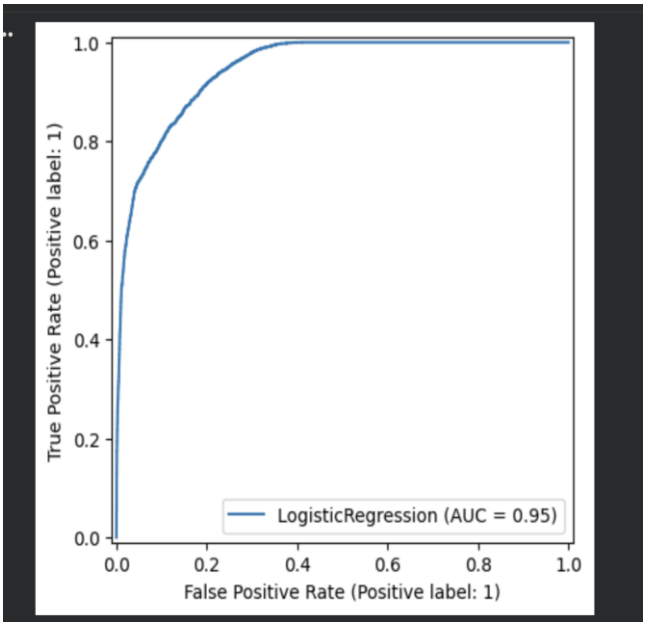
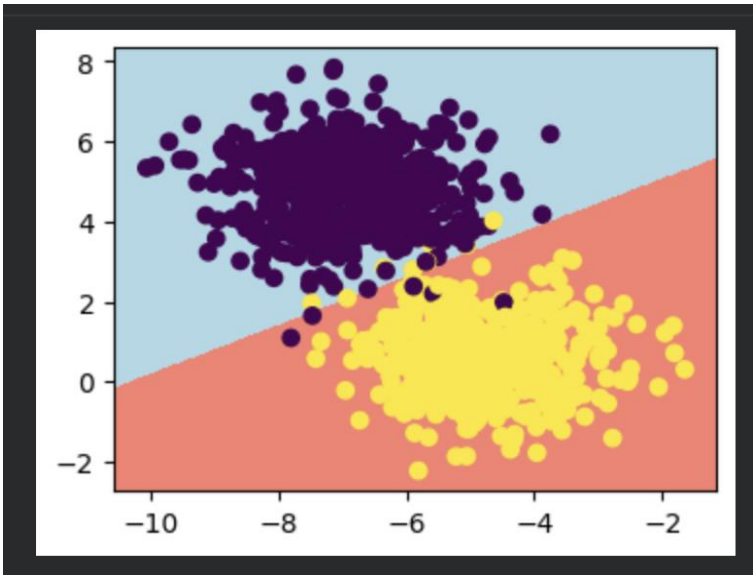


Figure 3. Decision Boundary



5. Discussion:

The decision boundary and ROC curve indicate that the logistic regression model performed well in distinguishing between days with and without rainfall. The model demonstrates strong predictive ability, especially shown by its high true-positive rate across various thresholds. However, the confusion matrix reveals that while the model is generally accurate, it still makes a noticeable number of misclassifications. This suggests that the model is not yet fully reliable and could benefit from additional refinement. Factors such as class imbalance, limited feature variety, or noise in the dataset may contribute to the remaining errors. Overall, the results are promising, but further improvements—such as incorporating more features, testing more advanced models, or tuning hyperparameters—could enhance predictive performance.

6. Conclusion:

The goal of this project was to develop a model capable of predicting whether rainfall would occur on a given day. Using weather data from multiple states, the logistic regression model demonstrated strong predictive performance, suggesting that the chosen features and large number of data points contributed to its accuracy. Overall, the project successfully produced a model that can forecast rainfall with a high degree of reliability.

However, there is still room for improvement. Future work could explore additional machine learning models such as decision trees and random forests, which may capture nonlinear relationships in the data and potentially improve predictive accuracy. Random forests could help reduce overfitting and provide more robust predictions. Another challenge encountered in this project was finding consistent and reliable data sources. With more time and resources, collecting a dedicated dataset or integrating higher-quality weather data could further enhance model performance.

7. Reference:

[1] A. Waqar (2024). USA Rainfall Prediction Dataset (2024-2025) [Dataset]. kaggle. https://www.kaggle.com/datasets/waqi786/usa-rainfall-prediction-dataset-2024-2025/data